Convenient Algebraic Programming with Coercive Subtyping

Henry Blanchette[1*] and Aaron Stump[2*]

[1]University of Maryland.
[2]Boston College.

*Corresponding author(s). E-mail(s): blancheh@umd.edu;
stumpaa@bc.edu;

Abstract

This paper presents a system for more convenient algebraic programming, a form of total functional programming where datatypes are defined as least fixed points of covariant signature functors, and recursive functions are defined as algebras. This approach requires the use of certain combinators to roll and unroll fixed points, and to convert algebras to functions which can be applied to recurse over data. These combinators are a burden for programming in this style. To alleviate that burden, in this paper we propose to insert them automatically, using coercive subtyping. The type checker generates subtyping constraints, which are then solved with the help of a custom logic programming engined called Chronolog. The subtyping axioms are given to Chronolog as rules, which uses them to solve the subtyping constraints. To avoid infinite applications of rules for unrolling signature functors, Chronolog provides a mechanism for suspending and resuming certain forms of goals. The subtyping axioms presented to Chronolog are an algorithmic version of the usual declarative rules for algebraic programming. We prove equivalence of the declarative and algorithmic rules, in Agda. We present examples of algebraic programs written in our system.

Keywords: strong functional programming, type-based termination, Mendler-style recursion

1 Introduction

Strong functional programming is a form of pure functional programming where all functions are statically required to terminate on all inputs. One way to achieve this is

through an algebraic approach: datatypes are least fixed points μF of signature functors F, and recursive functions are least fixed points $(\!|a|\!)$ of algebras a. The functions are called catamorphisms. The signature functor can be seen as describing one layer of the data. The least fixed point then describes data consisting of any finite number of layers. We may unroll μF to $F\ (\mu F)$, and roll it back again. Such transformations are needed to allow construction of data by applying constructors of the signature functor, and destruction by pattern-matching against those constructors. Algebras interpret the operations declared for one layer of the data, and catamorphisms then apply algebras across all the layers. There is no other mechanism for recursion or looping in the language. This leads to a strong guarantee: all functions written in this style are guaranteed to terminate.

One hindrance to algebraic programming is the need to apply the functions for rolling and unrolling signature functors, and to apply $(\!|-|\!)$ to obtain a recursive function from an algebra. In this paper, we remove this hindrance by automatically inserting those coercions wherever they are needed in code. This means that the programmer may write code that looks indistinguishable from regular pure-functional code, and yet falls within the algebraic style, thus assuring termination. Code which attempts to make disallowed recursive calls will still be rejected by the type checker.

To insert these functions automatically, we turn to the technique of coercive subtyping. With this approach, the type checker generates subtyping constraints that need to hold, in order for the term to be well-typed. The constraints are then processed by a constraint solver. Successful runs of the solver compute the coercions for the proven subtyping statements. These are then inserted in the code, at the places where the subtyping constraints were generated. The constraints may contain type variables, which should be instantiated during solving.

To implement this, we make use of a custom logic-programming engine called Chronolog. The subtyping axioms are given to Chronolog as rules. Chronolog then uses standard SLD resolution to search for proofs of the constraints, instantiating type variables during proof search via unification. The rules for rolling and unrolling signatures would, if handled naively, lead to diverging proof search, as functors could be unrolled unboundedly, and even rolling could diverge, if type variables are instantiated to functor applications, which then roll back to just a bare type variable again. To avoid this, Chronolog provides a mechanism to indicate that certain goals should be suspended. We identify syntactic forms of goals which should not be processed, on pain of divergence. These suspended goals may be resumed if they are instantiated enough during solving of other goals. If not, an outer loop of our algorithm handles them by simply equating the lower and upper bound of the subtyping constraints.

An important technical issue with this approach concerns the subtyping rules themselves. Declarative rules for subtyping generally are not algorithmic, due to the presence of a transitivity rule. This rules could be applied indefinitely during proof search, as it posits an intermediate type B for solving a constraint $A <: C$, and requiring subgoals $A <: B$ and $B <: C$. To handle this, one must devise an equivalent set of rules without transivity. The goal could be compared to cut-elimination in sequent calculus, where the cut rule is a form of transitivity of entailment. We identify (Section 5) an algorithmic set of subtyping rules, without transitivity, and prove

it equivalent to the declarative rules. This both ensures we can use logic programming (with our algorithmic rules) to solve these subtyping constraints, and also assures us that the algorithmic rules are indeed sound and complete. This proof of equivalence has been formalized and checked in Agda.

To summarize the contributions:

- We propose a new approach to total algebraic programming, where function calls for rolling and unrolling signature functors and turning algebras into catamorphisms are inferred by coercive subtyping.
- We realize this approach by generating subtyping constraints during typing, and solving them using a constraint solver. The solver is a logic-programming engine, with a custom mechanism to suspend goals that would other lead to divergence.
- We propose an algorithmic version of the subtyping rules for this theory, and prove it equivalent to the declarative ones. The proof is checked in Agda.

2 Motivating examples

[TODO for Henry]:

pick nice example
subtraction uses unroll, but not roll or cata length uses roll, but not unroll or cata
write an outer function that subtracts some quantity from the length of a list
here it is in Haskell here it is in algebraic style with coercions
Can we use the algebraic style with lighter syntax? This would make it more feasible in practice.

[TODO for Henry]:

Haskell version

(a) Haskell

[TODO for Henry]:

Algebraic version without coercions

(b) Algebraic

```
sub : Nat → Nat ⇒ Nat
sub = λ m . ω sub' n . γ n .
          { Zero() → m
          | Suc(n') → γ (unroll id) (sub' n') .
              { Zero() → (roll id) Zero()
              | Suc(m') → m' } }

subs : NatList ⇒ NatList → NatList
subs = ω subs' l1 . λ l2 . γ l1 .
          { Nil → Nil
          | Cons(h1, t1) → γ (unroll id) l2 .
              { Nil → (roll id) Nil
              | Cons(h2, t2) → (roll id) Cons((cata (sub h1)) h2, subs' t1 t2) } }
```

(c) Algebraic with coercions

Fig. 1: Implementations of `sub` and `subs`.

3 Syntax

[TODO for Henry]:

Write prose (maybe Aaron helping with prose)

$$
\begin{array}{rrcl}
\text{(Datatype Names)} & D & \in & \textbf{DatatypeNames} \\
\text{(Term variables)} & x & \in & \textbf{TermVars} \\
\text{(Coercion variables)} & r & \in & \textbf{CoercionVars} \\
\text{(Recursion Type variables)} & R & \in & \textbf{RecTypeVars} \\
\text{(Type variables)} & X, Y & \in & \textbf{TypeVars} \\
\text{(Contexts)} & \Gamma & ::= & \varnothing \mid \Gamma, (x : A) \mid \Gamma, (R : *) \\
\text{(Types)} & A, B & ::= & X \mid R \mid \mu D \mid D\ A \mid A \to B \mid A \Rightarrow B \\
\text{(Terms)} & a, b, f & ::= & x \mid D.k(\overline{a,}) \mid \lambda(x : A).b \mid (f\ a) \mid \omega f(x).b \mid \\
 & & & \gamma a.\{\overline{k(\overline{x,}) \mapsto b \mid}\} \mid \text{let } x = a \text{ in } b \mid \text{coerce } c\ a \\
\text{(Coercions)} & c, c_1, c_2 & ::= & r \mid \text{cata } c_1\ c_2 \mid \text{roll } c \mid \text{unroll } c \mid \text{cov}[D]\ c \mid \text{idData}[D] \mid \\
 & & & \text{subArr } c_1\ c_2 \mid \text{subAlg}[D]\ c
\end{array}
$$

4 Typing with Constraints

Type checking consists of constraint generation phase and a constraint solving phase. Constraint generation for a well-formed and well-scoped term a in context Γ is performed by applying the constraint generation rules presented in figure 2 recursively on the structure of a, which upon success produces a derivation of $\Gamma \vdash a : A \mid \mathcal{A} \implies \mathcal{C}$. This is a usual typing judgment augmented with a constraint problem $\mathcal{A} \implies \mathcal{C}$, where $\mathcal{A}$ is a set of assumed constraints and $\mathcal{C}$ is a set of required constraints. Each constraint in these sets is of the form $A <::^p B$, where p is a position of a sub-term of a. The system is designed such that a solution to $\mathcal{A} \implies \mathcal{C}$ specifies an insertion of coercions around sub-terms of a, where each coercion inserted at a position p corresponds to one of the original required constraints $A <::^p B$.

The constraint generation phase also performs type inference, however in contrast to usual type inference and checking procedures, this procedure cannot decide a term to be mal-typed. The constraint generation procedure will generate a constraint problem for any well-formed and well-scoped Γ, a, A. A derivation $\Gamma \vdash a : A \mid \mathcal{A} \implies \mathcal{C}$ merely states that a has type A *only if* the constraint problem $\mathcal{A} \implies \mathcal{C}$ is solvable.

Performing the constraint generation procedure on `sub : Nat → Nat ⇒ Nat` results in the following constrained typing judgment:

$$
\varnothing \vdash sub : \mu Nat \to Nat \Rightarrow \mu Nat \mid \{R <:: R\} \implies \{? \to ? \Rightarrow X_1 <:: \mu Nat \to Nat \Rightarrow
$$

$$
\mu Nat, R \to \mu Nat <:: R \to X_2, X_2 <:: Nat\ \mu Nat, Nat\ X_3 <:: X_1\}
$$

[TODO for Henry]:

Prose

The judgment for inferring subtyping constraints is $\Gamma \vdash a : A \mid \mathcal{A} \implies \mathcal{C}$, where $\mathcal{A}, \mathcal{C}$ are sets of constraints of the form $A <:: B$. This judgment states that in context Γ, the term a is inferred to have type A under the constraint that the assumptions $\mathcal{A}$ imply the requirements $\mathcal{C}$.

$$\textsc{InferVar}\ \frac{(x : A) \in \Gamma}{\Gamma \vdash x : A \mid \varnothing \implies \varnothing}$$

$$\textsc{InferCon}\ \frac{X \text{ fresh} \qquad k : (\overline{B_i},) \to D\ X \qquad (\forall i)\ \Gamma \vdash a_i : A_i \mid \mathcal{A}_{a_i} \implies \mathcal{C}_{a_i}}{\Gamma \vdash k(\overline{a_i},) : D\ X \mid \bigcup_i (\{A_i <:: B_i\} \cup \mathcal{A}_{a_i}) \implies \bigcup_i \mathcal{C}_{a_i}}$$

$$\textsc{InferLam}\ \frac{X \text{ fresh} \qquad \Gamma, (x : X) \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b}{\Gamma \vdash \lambda x.b : X \to B \mid \mathcal{A}_b \implies \mathcal{C}_b}$$

$$\textsc{InferApp}\ \frac{X \text{ fresh} \qquad \Gamma \vdash f : B \mid \mathcal{A}_f \implies \mathcal{C}_f \qquad \Gamma \vdash a : A \mid \mathcal{A}_a \implies \mathcal{C}_a}{\Gamma \vdash f\ a : X \mid \{B <:: A \to X\} \cup \mathcal{A}_f \cup \mathcal{A}_a \implies \mathcal{C}_f \cup \mathcal{C}_a}$$

$$\textsc{InferGamma}\ \frac{\begin{array}{c} X, Y \text{ fresh} \qquad \Gamma \vdash a : C \mid \mathcal{A}_a \implies \mathcal{C}_a \\ (\forall i)\ k_i : (\overline{A_{ij}},) \to D\ X \qquad (\forall i)\ \Gamma, \overline{x_{ij} : A_{ij}}, \vdash b_i : B_i \mid \mathcal{A}_{b_i} \implies \mathcal{C}_{b_i} \end{array}}{\Gamma \vdash \gamma a.\{\overline{k_i(\overline{x_{ij}},) \mapsto b_i}\ |\} : Y \mid \mathcal{A}_a \cup \bigcup_i \mathcal{A}_{b_i} \implies \{C <:: D\ X\} \cup \mathcal{C}_a \cup \bigcup_i (\{B_i <:: Y\} \cup \mathcal{C}_{b_i})}$$

$$\textsc{InferOmega}\ \frac{R, r \text{ fresh} \qquad \Gamma, (R : *), (r : R <:: R), (f : R \to A), (x : D\ R) \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b}{\Gamma \vdash \omega f(x).b : D \Rightarrow B \mid \mathcal{A}_b \cup \{R <:: R\} \implies \mathcal{C}_b}$$

$$\textsc{InferLet}\ \frac{\Gamma \vdash a : A \mid \mathcal{A}_a \implies \mathcal{C}_a \qquad \Gamma, (x : A) \vdash b : B \mid \mathcal{A}_b \implies \mathcal{C}_b}{\Gamma \vdash \text{let } x = a \text{ in } b : B \mid \mathcal{A}_a \cup \mathcal{A}_b \implies \mathcal{C}_a \cup \mathcal{C}_a}$$

Fig. 2: Constraint generation rules.

$$\textsc{VarCoe}\ \frac{(r : A <:: B) \in \Gamma}{\Gamma \vdash r : A <:: B} \qquad \textsc{ReflCoe}\ \frac{D\ \text{data}}{\Gamma \vdash \text{idData}[D] : \mu D <:: \mu D}$$

$$\textsc{ArrCoe}\ \frac{\Gamma \vdash c_1 : A' <:: A \qquad \Gamma \vdash c_2 : B <:: B'}{\Gamma \vdash \text{subArr}\ c_1\ c_2 : A \to B <:: A' <:: B'} \qquad \textsc{CovCoe}\ \frac{\Gamma \vdash c : A <:: A'}{\Gamma \vdash \text{cov}[D]\ c : D\ A <:: D\ A'}$$

$$\textsc{AlgCoe}\ \frac{\Gamma \vdash c : A <:: A'}{\Gamma \vdash \text{subAlg}[D]\ c : D \Rightarrow A <:: D \Rightarrow A'} \qquad \textsc{RollCoe}\ \frac{D\ \text{data} \qquad \Gamma \vdash c : A <:: \mu D}{\Gamma \vdash \text{roll}\ c : D\ A <:: \mu D}$$

$$\textsc{UnrollCoe}\ \frac{D\ \text{data} \qquad \Gamma \vdash c : \mu D <:: A}{\Gamma \vdash \text{unroll}\ c : D\ A <:: \mu D} \qquad \textsc{CataCoe}\ \frac{\Gamma \vdash c_1 : B <:: D \qquad \Gamma \vdash c_2 : A <:: C}{\Gamma \vdash \text{cata}\ c_1\ c_2 : D \Rightarrow B <:: B \to C}$$

Fig. 3: Coercion type checking rules.

$$\textsc{DataBool}\ \frac{}{\mathit{Bool}\ \text{data}} \qquad \textsc{Bool.True}\ \frac{}{\mathit{True} : () \to \mathit{Bool}\ A} \qquad \textsc{Bool.False}\ \frac{}{\mathit{False} : () \to \mathit{Bool}\ A} \qquad \textsc{DataNat}\ \frac{}{\mathit{Nat}\ \text{data}}$$

$$\textsc{Nat.Zero}\ \frac{}{\mathit{Zero} : () \to \mathit{Nat}\ A} \qquad \textsc{Nat.Suc}\ \frac{}{\mathit{Suc} : (A) \to \mathit{Nat}\ A} \qquad \textsc{DataNatList}\ \frac{}{\mathit{NatList}\ \text{data}}$$

$$\textsc{NatList.Nil}\ \frac{}{\mathit{Nil} : () \to \mathit{NatList}\ A} \qquad \textsc{NatList.Cons}\ \frac{}{\mathit{Cons} : (\mathit{Nat}, A) \to \mathit{NatList}\ A}$$

Fig. 4: Datatype constructor type inference rules.

5 Subtyping

We begin with the declarative subtyping rules corresponding to the coercions of Figure 3, and then propose an algorithmic version, where all meta-variables in the premises of the rule are already present in the conclusion, and the sizes of the constraints decrease. This ensures that SLD resolution will not diverge on ground constraints. We prove that the two sets of rules are equivalent.

5.1 Declarative subtyping

$$\textsc{Refl}\ \frac{D\ \text{data}}{\mu D <: \mu D} \qquad \textsc{Arr}\ \frac{A' <: A \quad B <: B'}{A \to B <: A' \to B'} \qquad \textsc{Cov}\ \frac{A <: A'}{D\ A <: D\ A'} \qquad \textsc{Alg}\ \frac{A <: A'}{D \Rightarrow A <: D \Rightarrow A'}$$

$$\textsc{Roll}\ \frac{}{D\ \mu D <: \mu D} \qquad \textsc{Unroll}\ \frac{}{\mu D <: D\ \mu D} \qquad \textsc{Cata}\ \frac{}{D \Rightarrow A <: \mu D \to A} \qquad \textsc{Trans}\ \frac{A <: A' \quad A' < A''}{A <: A''}$$

Fig. 5: Declarative subtyping rules.

Figure 5 shows the rules for declarative subtyping $A <: B$. We have reflexivity rule Refl for datatypes. Arr is the usual subtyping rule for function types, which is contravariant in the domain and covariant in the codomain. Cov expresses that the signature functor is indeed a (covariant) functor. Alg expresses that algebra types $D \Rightarrow A$ are covariant in their carriers A. Roll and Unroll together express that μF is equivalent to $F\ (\mu F)$. Cata expresses that an algebra can be coerced to a function, namely the algebra's catamorphism. Finally, there is the Trans rule, completing the definition of subtyping as a partial order.

5.2 Algorithmic subtyping

$$\textsc{Refl}\ \frac{D\ \text{data}}{\mu D <:: \mu D} \qquad \textsc{Arr}\ \frac{A' <:: A \quad B <:: B'}{A \to B <:: A' \to B'} \qquad \textsc{Cov}\ \frac{A <:: A'}{D\ A <:: D\ A'}$$

$$\textsc{Alg}\ \frac{A <:: A'}{D \Rightarrow A <:: \mu D \Rightarrow A'} \qquad \textsc{Roll}\ \frac{A <:: \mu D}{D\ A <:: \mu D} \qquad \textsc{Unroll}\ \frac{\mu D <:: A}{\mu D <:: D\ A} \qquad \textsc{Cata}\ \frac{B <:: D \quad A <:: C}{D \Rightarrow A <:: B \to C}$$

Fig. 6: Algorithmic subtyping rules.

Figure 6 presents the rules for algorithmic subtyping $A <:: B$. We use similar names as for the declarative rules, but critically, the algorithmically problematic Trans rule is not present. A consequence of this is that now several rules that did not have premises in the declarative formulation, here do. For example, compare the declarative (on the

left) and algorithmic CATA rules:

$$\frac{}{D \Rightarrow A <: \mu D \to A} \qquad \frac{B <:: D \quad A <:: C}{D \Rightarrow A <:: B \to C}$$

The algorithmic rule needs to take into account subtyping relationships between the components in the conclusion, because there is no rule for transitivity, which could allow those relationships to be taken into account after this rule is applied. In effect, transitivity needs to be built into all the other rules. We see a similar change from the declarative to the algorithmic versions of the ROLL and UNROLL rules, for rolling and unrolling the signature functor.

Lemma 1 (Decrease) *For all rules of Figure 6, the sizes of the constraints in the premises are less than the size of the constraint in the conclusion.*

Theorem 2 (Decidability) *It is decidable whether or not two types are in the algorithmic subtyping relation.*

Proof Bottom-up application of the rules to ground types is guaranteed to terminate, by Lemma 1. Therefore, proof search may be used as a decision procedure for algorithmic subtyping of ground types. □

5.3 Equivalence of declarative and algorithmic subtyping

While Theorem 2 ensures that we can test ground subtyping constraints using proof search, we must confirm that the algorithmic rules are indeed a correct version of the declarative ones. For this, we prove the following theorems.

Theorem 3 (Soundness) *$A <:: B$ implies $A <: B$.*

Theorem 4 (Completeness) *$A <: B$ implies $A <:: B$.*

The sets of rules are quite similar, so these proofs are not onerous. They do depend on the following inductively proven lemmas.

Lemma 5 (roll1 (declarative)) *If $A <: \mu D$, then $D\ A <: \mu D$.*

Lemma 6 (roll2 (declarative)) *If $\mu D <: A$, then $\mu D <: D\ A$.*

Lemma 7 (refl (algorithmic)) *$A <:: A$.*

Lemma 8 (trans (algorithmic)) *If $A <:: B$ and $B <:: C$ then $A <:: C$.*

```
data Tp : Set where
  D_ : Tp → Tp
  _⟶_ : Tp → Tp → Tp
  D⇒ : Tp → Tp
  μD : Tp

infixr 9 _⟶_
infixr 10 D_
```

Fig. 7: Agda definition of types

Lemma 9 (roll (algorithmic)) $D\ \mu D <: \mu D$

Lemma 10 (unroll (algorithmic)) $\mu\ D <: D\ \mu D$

5.4 Formalization in Agda

We formalize the above development in Agda as follows. Types are represented by the Agda datatype `Ty` in Figure 7. For simplicity, we have single signature functor `D` and its least fixed-point `μD` (note that this is a single identifier in Agda, not an application of a μ operator to `D`). Algebra types $D \Rightarrow A$ are represented just as `D⇒ A`. Note that `D⇒` is also a single symbol in Agda, like `μD`. Arrow types are represented $A \longrightarrow B$ (as other arrow notations are already in use). The figure declares some fixity information, for lighter notation where these operators are used later.

Figures 8 and 10 show the representations of the declarative and algorithmic subtyping systems, respectively. We follow a standard encoding strategy, where each set of rules is represented as an indexed inductive type. Each rule becomes a constructor of the type. For example, we have, for both types, `Refl` of type `μD <: μD`, which corresponds exactly to the `Refl` rules of Figures 5 and 6. Conveniently, Agda allows reusing constructor names across datatypes. Dependent typing is used for quantification. For example, the `Cata` constructor for declarative subtyping (in Figure 8) has type

```
{X : Tp} → D⇒ X <: μD ⟶ X
```

This is a dependent function type, stating that give `X : Tp`, the `Cata` constructor will return a proof of `D⇒ X <: μD ⟶ X`, expressing the coercion from an algebra with carrier `X` to its catamorphism. Finally, the statements of soundness and completeness are shown in Figure **??**. We omit the proofs of these and the lemmas listed in Section 5.3.

The total number of lines for all the Agda code is under 150. This shows that the proof burden of the approach is very manageable. Part of the reason for this is that the types do not contain bound variables, and so the technical overhead associated with reasoning in the presence of those does not arise.

```
infix 5 _<:_

data _<:_ : Tp → Tp → Set where
  Refl : μD <: μD
  Arr : {T1 T2 T1' T2' : Tp} →
        T1' <: T1 →
        T2 <: T2' →
        T1 ⟶ T2 <: T1' ⟶ T2'
  Roll : D μD <: μD
  Unroll : μD <: D μD
  Cov : {T1 T2 : Tp} → T1 <: T2 → D T1 <: D T2
  Trans : {T1 T2 T3 : Tp} → T1 <: T2 → T2 <: T3 → T1 <: T3
  Cata : {X : Tp} → D⇒ X <: μD ⟶ X
  Alg : {T1 T2 : Tp} → T1 <: T2 → D⇒ T1 <: D⇒ T2
```

Fig. 8: Agda definition of declarative subtyping

```
infix 5 _<::_

data _<::_ : Tp → Tp → Set where
  Refl : μD <:: μD
  Arr : {T1 T2 T1' T2' : Tp} →
        T1' <:: T1 →
        T2 <:: T2' →
        T1 ⟶ T2 <:: T1' ⟶ T2'
  Roll : {T : Tp} → T <:: μD → D T <:: μD
  Unroll : {T : Tp} → μD <:: T → μD <:: D T
  Cov : {T1 T2 : Tp} → T1 <:: T2 → D T1 <:: D T2
  Alg : {T1 T2 : Tp} → T1 <:: T2 → D⇒ T1 <:: D⇒ T2
  Cata : {T T1 T2 : Tp} →
         T1 <:: μD →
         T <:: T2 →
         D⇒ T <:: T1 ⟶ T2
```

Fig. 9: Agda definition of algorithmic subtyping

6 Implementation with Chronolog

[TODO for Henry]:

Prose

```
Soundness    : {T T' : Tp} → T <:: T' → T <: T'
Completeness : {T T' : Tp} → T <: T'  → T <:: T'
```

Fig. 10: Agda statements of soundness and completeness

7 Returning to the Examples

[TODO for Henry]:

Prose remind the version with coercions show without coercions

8 Related Work

Turner introduced the term *strong functional programming*, presented several arguments in favor of the approach, and proposed an *elementary* way to realize such a language, in the form of static checks for structural recursion [1]. Mendler proposed a recursor using an abstract type to ensure that recursive calls are permitted only on subdata for an inductive type [2]. The algebraic approach has been applied for modular implementation of interpreters [3].

Charity is a strong functional programming language based on categorical abstractions for initial algebras and final coalgebras [4]. Tofu uses the Calculus of Constructions as a language for defining terminating functions, again based on categorical ideas [5]. Charity and Tofu both support coinductive datatypes, which we did not study here.

9 Conclusion and future work

We have seen how total algebraic programming can be made more feasible by inferring the basic coercions necessary for the method. These are for rolling and unrolling signature functors, and for turning an algebra into its catamorphism. We considered an approach to this problem, where type inference generates subtyping constraints, whose solutions correspond to the needed coercions. Our constraint solver uses a logic-programming engine to process constraints with respect to an algorithmic version of the subtyping rules. This algorithmic subtyping relation was proved equivalent to the declarative one, in Agda.

Future work includes extending this paradigm further, to richer forms of algebraic programming. One form that has been proposed, as a powerful generalization of the basic structural recursion captured by Mendler algebras, is that of recursive coalgebras [6]. This formulation encompasses divide-and-conquer algorithms, which are beyond the scope of structural recursion. A different approach to divide-and-conquer recursion has also been considered by Abreu et al. [7]. That approach requires algebras to make various function calls to pass between various abstract types. These would be excellent candidates for inference using coercive subtyping, as we proposed in this

paper. It would also be interesting to infer coercions for coalgebraic programming with coinductive types.

References

[1] Turner, D.A.: Elementary Strong Functional Programming. In: Proceedings of the First International Symposium on Functional Programming Languages in Education. FPLE '95, pp. 1–13. Springer, Berlin, Heidelberg (1995)

[2] Mendler, N.P.: Inductive types and type constraints in the second-order lambda calculus. Annals of Pure and Applied Logic **51**(1), 159–172 (1991)

[3] Swierstra, W.: Data Types à La Carte. J. Funct. Program. **18**(4), 423–436 (2008)

[4] Cockett, R., Fukushima, T.: About Charity. Yellow Series Report No. 92/480/18, Department of Computer Science, The University of Calgary (June 1992)

[5] Widemann, B.T.: Tofu - towards practical church-style total functional programming. In: Workshop der GI-Fachgruppe Programmiersprachen und Rechenkonzepte, Bad Honnef, 3.-5. Mai (2010). Available at https://www-ps.informatik.uni-kiel.de/fg214/Honnef2010/TechnischerBericht1010.pdf

[6] Capretta, V., Uustalu, T., Vene, V.: Recursive coalgebras from comonads. Inf. Comput. **204**(4), 437–468 (2006) https://doi.org/10.1016/J.IC.2005.08.005

[7] Abreu, P., Delaware, B., Hubers, A., Jenkins, C., Morris, J.G., Stump, A.: A type-based approach to divide-and-conquer recursion in coq. Proc. ACM Program. Lang. **7**(POPL), 61–90 (2023) https://doi.org/10.1145/3571196